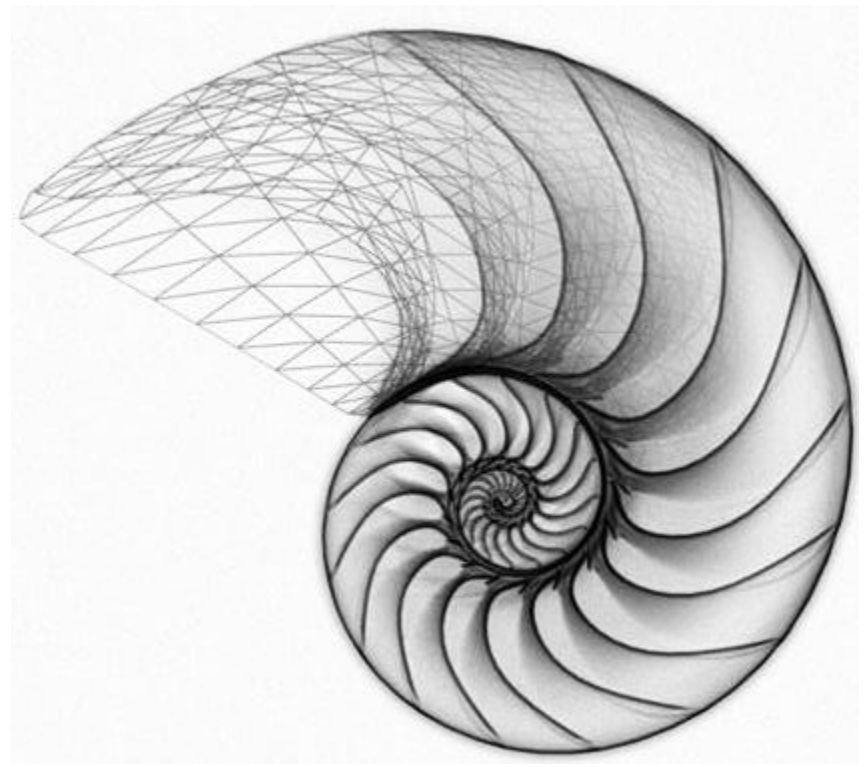




Lecture 01

Pointers

CSE225: Data Structures and Algorithms

What is a Pointer?

Variables allow the programmer to directly manipulate the data in memory.

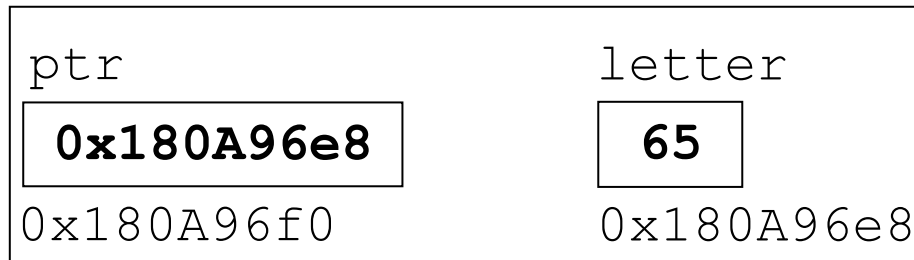
A pointer variable, however, does not store a value but store the **address of the memory** space which contains the value i.e. **it directly points to a specific memory address.**

Why would we want to use pointers?

- To call a function with memory addresses as parameters so that it can change values of variables created outside the function.
- To create a dynamic data structure which can grow larger or smaller as necessary.

Pointers

A pointer is a variable that contains the address of another variable.

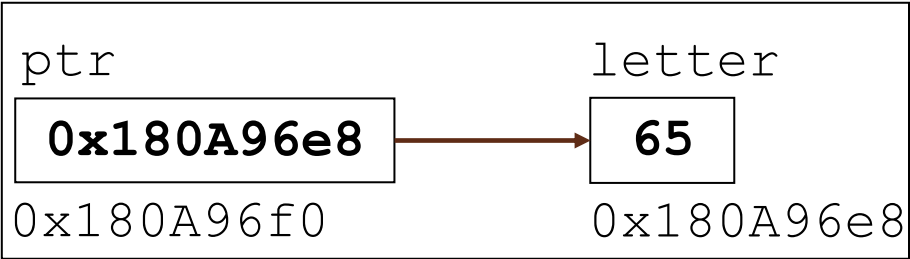


The variable **ptr** contains the address of **letter**

	address	content
	0x00000000	
	0x00000001	
	.	
	.	
	.	
letter	0x180A96e8	65
	0x180A96e9	
ptr	0x180A96f0	0x180A96e8
	0x180A96f1	
	0x180A96f2	
	0x180A96f3	
	.	
	.	

Pointers

A pointer is a variable that contains the address of another variable.
We say that a pointer points/references another variable.



The variable **ptr** contains the address of **letter**

	address	content
	0x00000000	
	0x00000001	
	.	
	.	
	.	
letter	0x180A96e8	65
	0x180A96e9	
ptr	0x180A96f0	0x180A96e8
	0x180A96f1	
	0x180A96f2	
	0x180A96f3	
	.	
	.	

Pointer Declaration

General Format:

```
data_type *pointer_name;
```

A pointer declaration such as,

```
int *numberPtr;
```

- declares **numberPtr** as a variable that **points to an integer variable**. Its content is a **memory address**.

The ***** indicates that the variable being declared is a pointer variable instead of a normal variable.

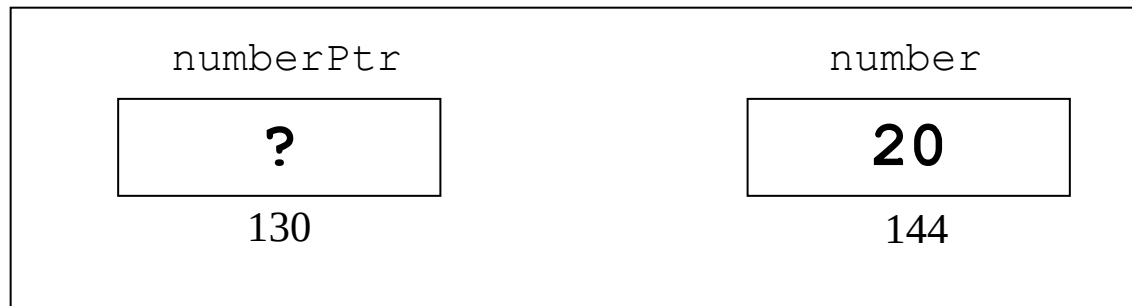
Pointer Declaration

Consider the following declaration

```
int *numberPtr, number = 20;
```

In this case, two memory address have been reserved, associated with the names **numberPtr** and **number**.

The value in variable **number** is of type integer, and the value in variable **numberPtr** is an address for another memory location.



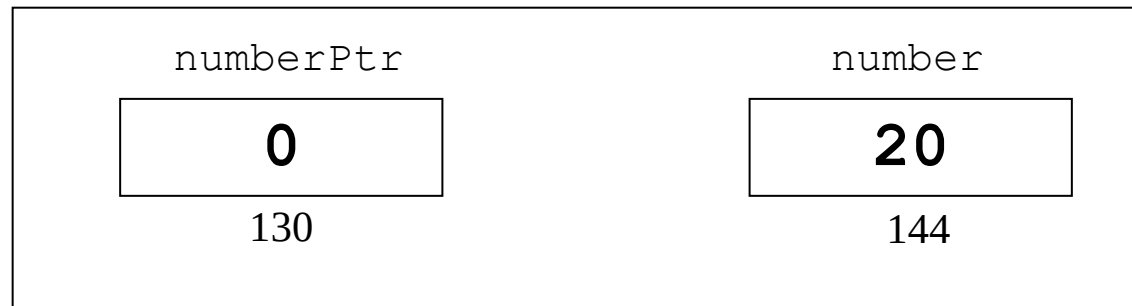
Pointer Initialization

To prevent the pointer from pointing to a random memory address, it is advisable that the pointer is initialized to **NULL (the value 0)** or **an address** before being used.

A pointer with the value **NULL**, points to nothing.

Initializing a pointer to 0 is equivalent to initializing a pointer to `NULL`, but `NULL` is preferred.

```
int *numberPtr = NULL, number = 20;
```



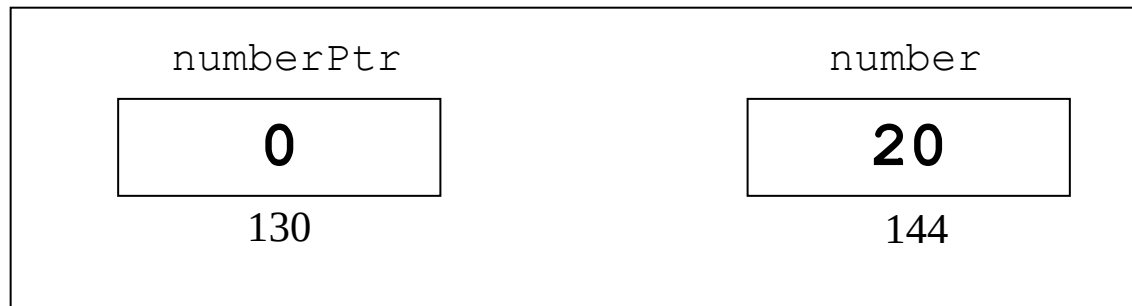
Pointer Operator (& and *)

When a pointer is created, it does not point to any valid memory address. Therefore, we need to assign a variable's address to it

- using the **&** operator (**referencing operator/ address-of operator**).

Look at this example:

```
int *numberPtr, number = 20;  
numberPtr = NULL;
```



Pointer Operator (& and *)

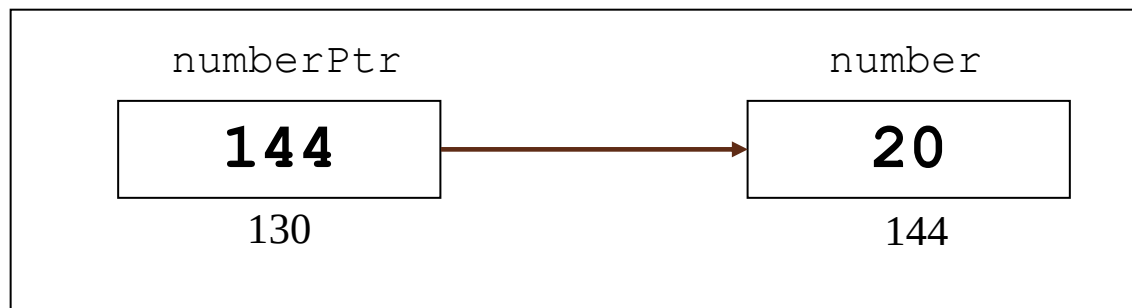
When a pointer is created, it does not point to any valid memory address. Therefore, we need to assign a variable's address to it

- using the **&** operator (**referencing operator/ address-of operator**).

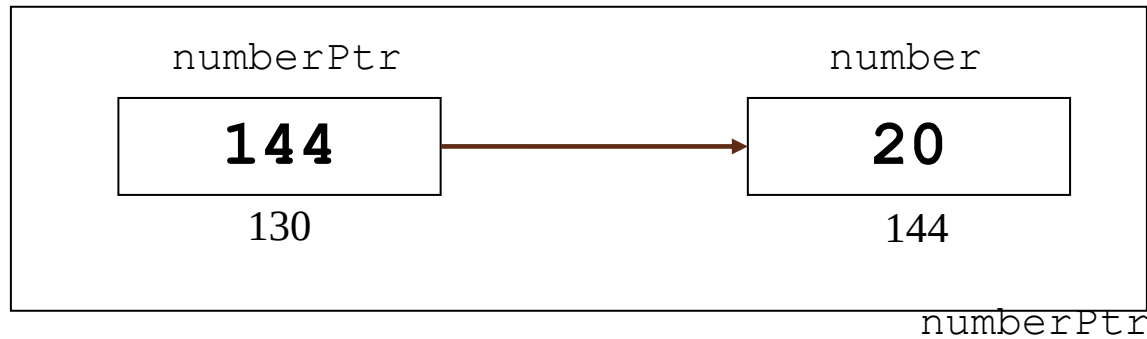
Look at this example:

```
int *numberPtr, number = 20;  
numberPtr = NULL;  
numberPtr = &number;
```

The statement **numberPtr = &number** assigns the address of the variable **number** to a pointer variable **numberPtr**. Variable **numberPtr** is then said as to “**point to**” variable **number**.



Pointer Operator (& and *)



address	content
0	
1	
.	
.	
.	
130	144
131	
132	
133	
.	
.	
144	20
145	
146	
147	
.	
.	

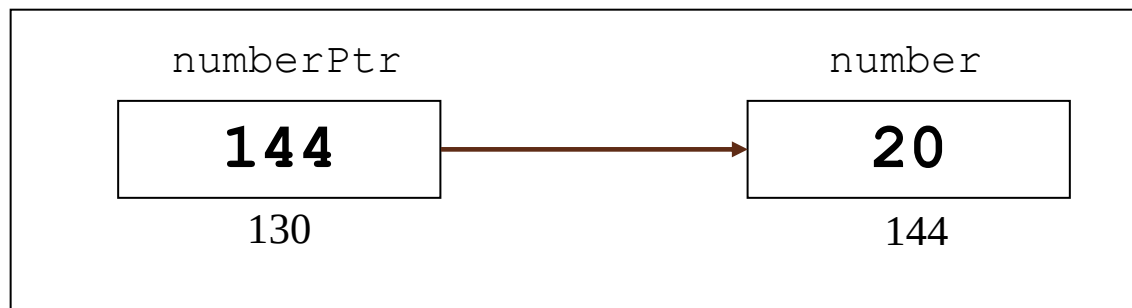
Pointer Operator (& and *)

After a pointer is assigned a particular address, the value at the pointed address can be accessed/modified

- using the * operator (**dereferencing operator/ value-at operator**).

Look at this example:

```
int *numberPtr, number = 20;  
numberPtr = NULL;  
numberPtr = &number;
```



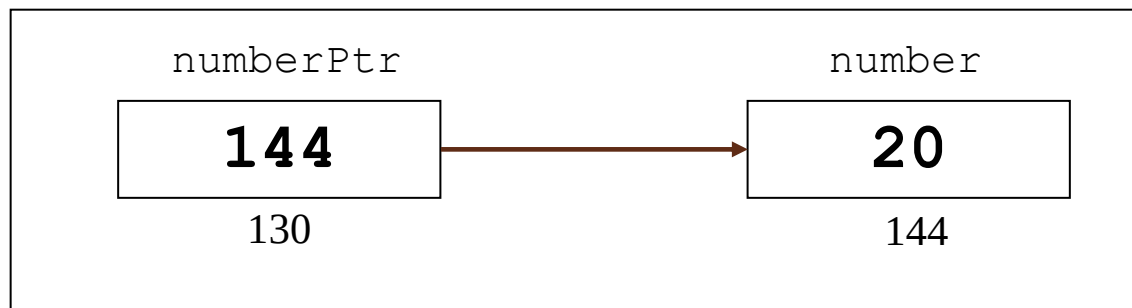
Pointer Operator (& and *)

After a pointer is assigned a particular address, the value at the pointed address can be accessed/modified

- using the * operator (**dereferencing operator/ value-at operator**).

Look at this example:

```
int *numberPtr, number = 20;  
numberPtr = NULL;  
numberPtr = &number;  
*numberPtr = 16;
```



Pointer Operator (& and *)

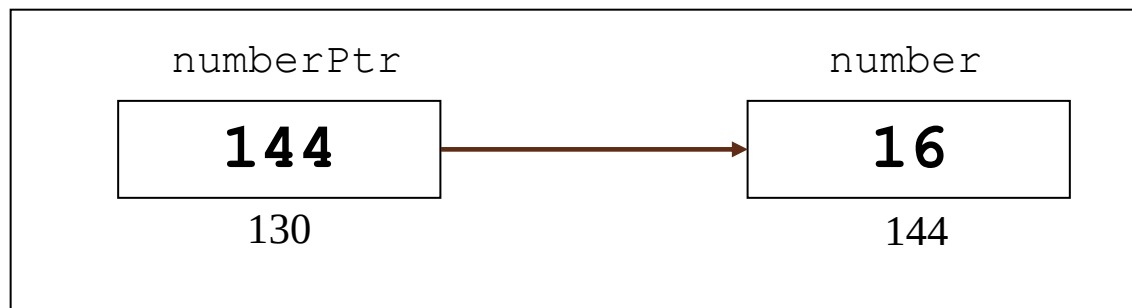
After a pointer is assigned a particular address, the value at the pointed address can be accessed/modified

- using the * operator (**dereferencing operator/ value-at operator**).

Look at this example:

```
int *numberPtr, number = 20;  
numberPtr = NULL;  
numberPtr = &number;  
*numberPtr = 16;
```

The statement ***numberPtr = 16** changes the content at the address 144 from 20 to 16.



Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(x,y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(x,y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Output:

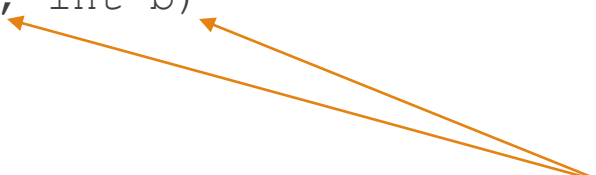
Before swap function: x = 5, y = 10

After swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;
```

```
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```



Local variables (get destroyed after function ends, no effect on x and y inside main)

```
int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(x,y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Output:

Before swap function: x = 5, y = 10

After swap function: x = 5, y = 10

Parameter Passing by Pointer

Declare the parameters of `swap` as pointer variables so that they can contain addresses.

```
void swap(int *addr1, int *addr2)
```

We will place the addresses of `x` and `y` into `addr1` and `addr2`, respectively.

```
swap(&x, &y) ;
```

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    ➡ int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Output:

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;
```

```
void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}
```

```
int main(void)
{
```

```
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

5

y

10

Output:

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;
```

```
void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}
```

```
int main(void)
{
```

```
    int x = 5, y = 10;
```

x

5

y

10

```
    ➔ cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
```

```
}
```

Output:

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    ➡ cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

5

y

10

Output:

Before swap function:

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;
```

```
void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}
```

```
int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    ➔ cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

5

y

10

Output:

Before swap function:

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    ➔ cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

5

y

10

Output:

Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    → swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

5

y

10

Output:

Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;
```

```
void swap(int *addr1, int *addr2)
{
    ➡ int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}
```

```
int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    ➡ swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

5

y

10

Output:

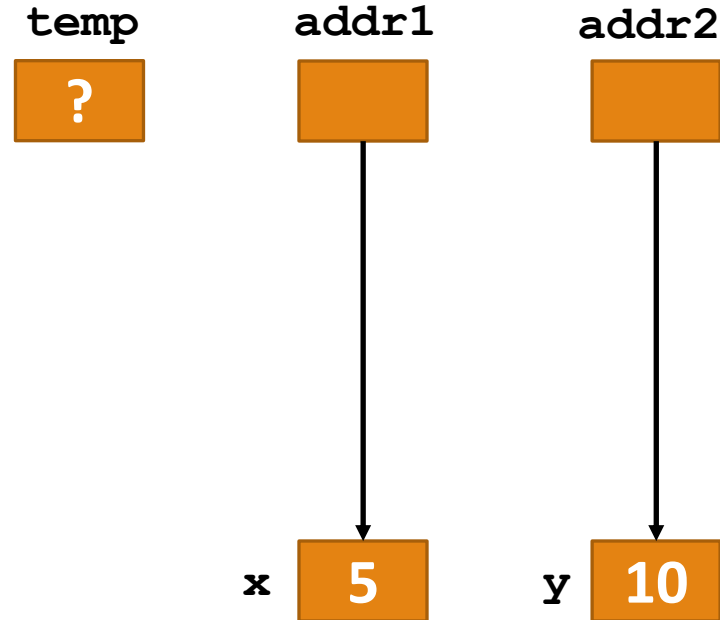
Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    ➡ int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    ➡ swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```



Output:

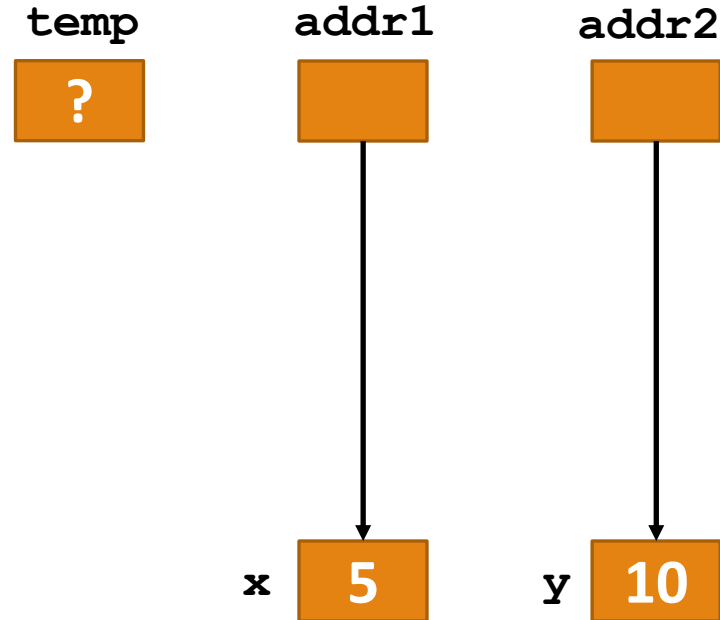
Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    ➡ temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    ➡ swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```



Output:

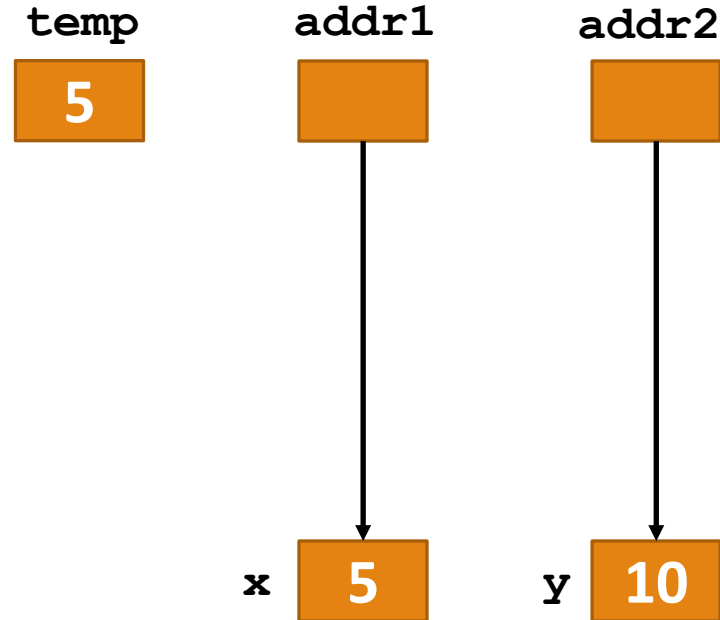
Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    ➡ temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    ➡ swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```



Output:

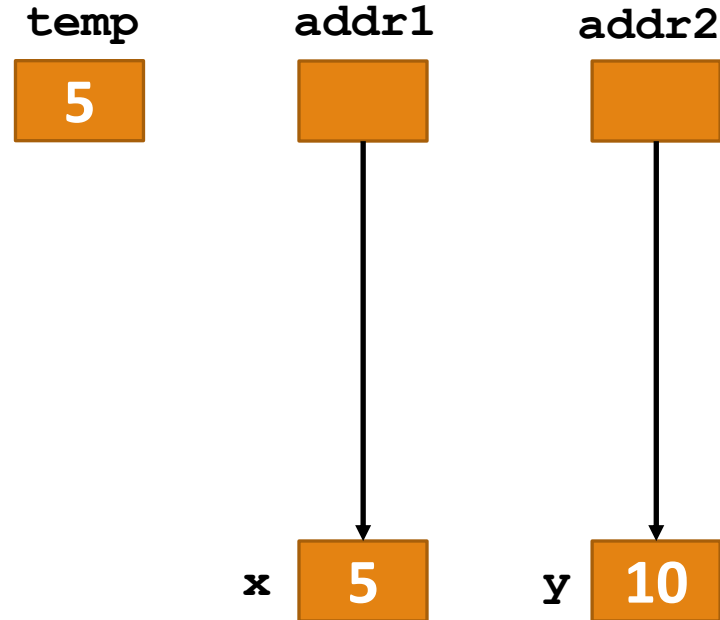
Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    → *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    → swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```



Output:

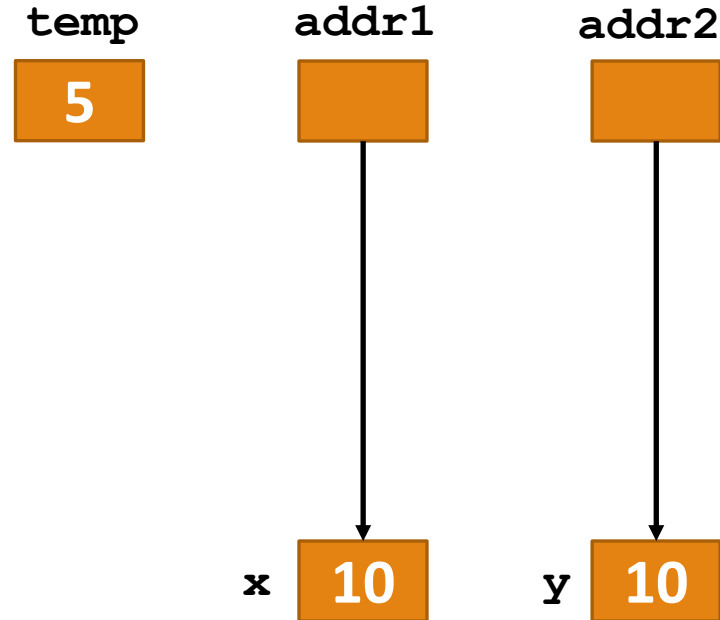
Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    → *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    → swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```



Output:

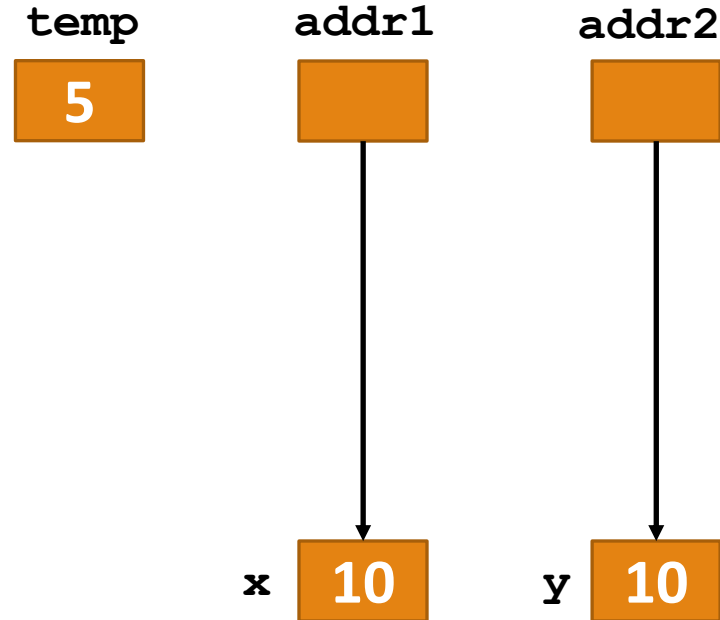
Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    → *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    → swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```



Output:

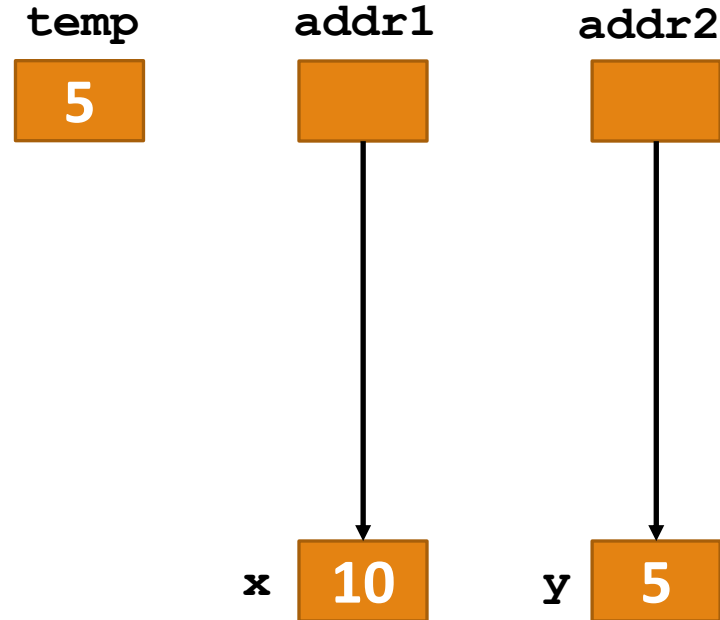
Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    → *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    → swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```



Output:

Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    → swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

10

y

5

Output:

Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

10

y

5

➡

Output:

Before swap function: x = 5, y = 10

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

10

y

5

➡

Output:

Before swap function: x = 5, y = 10

After swap function:

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

10

y

5



Output:

Before swap function: x = 5, y = 10
After swap function:

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    ➡ cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

10

y

5

Output:

Before swap function: x = 5, y = 10

After swap function: x = 10, y = 5

Parameter Passing by Pointer

```
#include <iostream>
using namespace std;

void swap(int *addr1, int *addr2)
{
    int temp;
    temp = *addr1;
    *addr1 = *addr2;
    *addr2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(&x, &y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

x

10

y

5



Output:

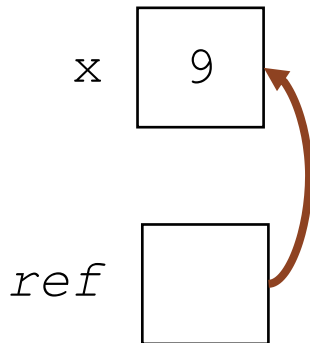
Before swap function: x = 5, y = 10

After swap function: x = 10, y = 5

Reference Variables

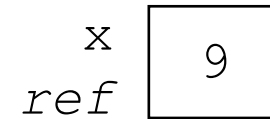
A reference is an additional name to an existing memory location

Pointer:



```
int x=9;  
int *ref;  
ref = &x;
```

Reference:



```
int x = 9;  
int &ref = x;
```


Reference Variables

A reference variable serves as an alternative name for an object.

```
int m = 10;  
int &j = m;    // j is a reference variable  
cout << "value of m = " << m << endl; //print 10  
j = 18;  
cout << "value of m = " << m << endl; // print  
18
```

Parameter Passing by Reference

```
#include <iostream>
using namespace std;

void swap(int &ref1, int &ref2)
{
    int temp;
    temp = ref1;
    ref1 = ref2;
    ref2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(x,y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Parameter Passing by Reference

```
#include <iostream>
using namespace std;

void swap(int &ref1, int &ref2)
{
    int temp;
    temp = ref1;
    ref1 = ref2;
    ref2 = temp;
}

int main(void)
{
    int x = 5, y = 10;
    cout << "Before swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    swap(x,y);
    cout << "After swap function: ";
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Output:

Before swap function: x = 5, y = 10

After swap function: x = 10, y = 5